

Python Functions to Filter — Complete Training Notes

Functions • Parameters & Arguments • print vs return • Prime Numbers • Iterables & Iterators • Generators & yield • Fibonacci • map() • Lambda • filter() • reduce() overview

Purpose: A clean, detailed version of the class notes with corrected explanations, syntax, code, outputs, internal working, real-world use cases, common mistakes, and interview revision.

1. Functions in Python

1.1 What is a Function?

A **function** is a reusable block of code designed to perform a specific task. Instead of rewriting the same logic many times, we define the logic once and call the function whenever needed.

```
def greet():  
    print("Hello!")  
  
greet()  
greet()  
  
# Output:  
# Hello!  
# Hello!
```

Functions improve code reuse, readability, testing, maintenance, and modularity.

1.2 Basic Syntax

```
def function_name(parameters):  
    # Function logic  
    return result
```

def starts the definition. The function name identifies the function. Parameters receive input. The indented block contains the logic. **return** sends a result back to the caller.

1.3 Parameters vs Arguments

```
def add_numbers(num1, num2):  
    return num1 + num2  
  
result = add_numbers(5, 6)  
print(result)  
  
# Output:  
# 11
```

num1 and **num2** are parameters. **5** and **6** are arguments—the actual values passed to the function.

1.4 Repeated Logic with Different Inputs

```
def multiplication_values(n):  
    for i in range(1, n + 1):  
        print(i * n)  
  
multiplication_values(6)
```

```
# Output:  
# 6  
# 12  
# 18  
# 24  
# 30  
# 36
```

`range(1, n + 1)` with `n = 6` produces values 1 through 6 because the stop value is excluded.

Correction: This produces multiples only through $n \times n$. A standard 1-to-10 multiplication table should use `range(1, 11)`.

1.5 Better Multiplication Table Example

```
def multiplication_table(number, limit=10):  
    for i in range(1, limit + 1):  
        print(f"{number} x {i} = {number * i}")  
  
multiplication_table(6)  
  
# Output starts:  
# 6 x 1 = 6  
# 6 x 2 = 12  
# ...  
# 6 x 10 = 60
```

2. print() vs return

`print()` displays information on the screen. `return` sends a value back from a function so that the program can store and reuse it.

2.1 print() Only Displays

```
def add_and_print(a, b):  
    print(a + b)  
  
result = add_and_print(5, 6)  
print(result)  
  
# Output:  
# 11  
# None
```

The function prints 11, but because it has no explicit `return` statement, its returned result is `None`.

2.2 return Sends Back a Reusable Value

```
def add_and_return(a, b):  
    return a + b  
  
result = add_and_return(5, 6)  
  
print(result)  
print(result + 9)  
  
# Output:  
# 11  
# 20
```

Feature	print()	return
Main purpose	Display a value	Send a result to the caller
Can function result be stored in a variable	NO	Yes
Can it be reused in calculations	Not as the returned result	Yes
No explicit return	Function returns None	—

2.3 Passing One Function's Result to Another

```
def calculate_age(current_year, birth_year):
    return current_year - birth_year

def age_after_five_years(age):
    return age + 5

age = calculate_age(2026, 2000)
future_age = age_after_five_years(age)

print(age)
print(future_age)

# Output:
# 26
# 31
```

The output of one function becomes the input of another. This is a key reason to use return.

3. Example: Prime Number Function

A prime number is greater than 1 and has exactly two positive divisors: 1 and itself.

```
def is_prime(number):
    if number < 2:
        return False

    for divisor in range(2, int(number ** 0.5) + 1):
        if number % divisor == 0:
            return False

    return True

print(is_prime(7))
print(is_prime(10))

# Output:
# True
# False
```

If any tested divisor divides the number exactly, the function immediately returns False. If no divisor is found, it returns True.

4. Iterables and Iterators

4.1 Iterable

An **iterable** is an object whose elements can be visited one at a time. Strings, lists, tuples, sets, dictionaries, and ranges are common iterables.

```

text = "HELLO"

for character in text:
    print(character)

# Output:
# H
# E
# L
# L
# O

```

4.2 Iterating with Indexes

```

text = "HELLO"

for i in range(len(text)):
    print(text[i])

# Output:
# H
# E
# L
# L
# O

```

`i` contains index values. `text[i]` accesses the actual character.

4.3 Iterator

An **iterator** is an object that returns one item at a time and remembers its current position.

```

text = "HELLO"
iterator = iter(text)

print(next(iterator)) # H
print(next(iterator)) # E
print(next(iterator)) # L
print(next(iterator)) # L
print(next(iterator)) # O

```

4.4 StopIteration

```

text = "HI"
iterator = iter(text)

print(next(iterator)) # H
print(next(iterator)) # I

# Calling next(iterator) again raises:
# StopIteration

```

A for loop automatically handles the iterator and stops when `StopIteration` occurs.

4.5 Conceptual Internal Working of a for Loop

```

iterator = iter(iterable)

while True:
    try:
        item = next(iterator)
        # Execute loop body
    except StopIteration:
        break

```

This is a conceptual explanation of the iterator protocol. Normally, use a regular for loop rather than manually rewriting it.

Term	Meaning
Iterable	An object that can provide an iterator
Iterator	Returns one item at a time and remembers state
iter()	Creates or retrieves an iterator
next()	Requests the next item
StopIteration	Signals that iteration is complete

5. Generators and yield

5.1 What is a Generator?

A **generator** is a special iterator commonly created by a function containing `yield`. It produces values lazily—one at a time when requested—instead of building the complete output collection first.

Correction: A generator still uses memory to preserve its execution state and local variables. Its advantage is that it usually does not keep the entire generated sequence in memory at once.

5.2 return vs yield

return	yield
Ends the current function execution	Pauses generator execution
Returns a result	Produces the next value
Normal function behavior	Generator behavior
A new call starts a new execution	Iteration resumes from saved state

5.3 Cube Generator

```
def generate_cubes(n):
    for number in range(n):
        yield number ** 3

for cube in generate_cubes(5):
    print(cube)

# Output:
# 0
# 1
# 8
# 27
# 64
```

The generator produces one cube, pauses, and continues when the next value is requested.

5.4 Manual Generator Iteration

```
def generate_cubes(n):
    for number in range(n):
        yield number ** 3

generator = generate_cubes(3)
```

```
print(next(generator)) # 0
print(next(generator)) # 1
print(next(generator)) # 8

# Another next(generator) raises StopIteration
```

5.5 Lazy Execution

```
def demo():
    print("Starting")
    yield 1
    print("Continuing")
    yield 2

g = demo()

print(next(g))
print(next(g))

# Output:
# Starting
# 1
# Continuing
# 2
```

Calling `demo()` creates the generator object. The function body progresses only as values are requested.

5.6 Fibonacci Generator

```
def fibonacci(n):
    a, b = 1, 1

    for _ in range(n):
        yield a
        a, b = b, a + b

for number in fibonacci(10):
    print(number, end=" ")

# Output:
# 1 1 2 3 5 8 13 21 34 55
```

The generator yields the current `a`, then updates the pair. The statement `a, b = b, a + b` performs simultaneous assignment.

Correction: The sequence progresses 1, 1, 2, 3, 5, 8—not '3 + 5 = 5'. Each new value is the sum of the previous two.

5.7 Where Generators Are Useful

Generators are useful for large files, streaming data, large numerical sequences, database rows, data pipelines, and any situation where processing one item at a time is more efficient than constructing a full list.

6. map()

6.1 Meaning

`map()` applies a function to every item of an iterable and returns a lazy map iterator.

```
def square(number):
    return number ** 2
```

```
numbers = [1, 2, 3, 4]

result = map(square, numbers)

print(list(result))

# Output:
# [1, 4, 9, 16]
```

Each input is passed through the same function: $1 \rightarrow 1$, $2 \rightarrow 4$, $3 \rightarrow 9$, and $4 \rightarrow 16$.

6.2 Syntax

```
map(function, iterable)
```

6.3 Fahrenheit to Celsius

```
def fahrenheit_to_celsius(temp):
    return (temp - 32) * 5 / 9

fahrenheit_values = [32, 68, 86, 104]

celsius_values = list(
    map(fahrenheit_to_celsius, fahrenheit_values)
)

print(celsius_values)

# Output:
# [0.0, 20.0, 30.0, 40.0]
```

6.4 Celsius Back to Fahrenheit

```
def celsius_to_fahrenheit(temp):
    return temp * 9 / 5 + 32

fahrenheit_again = list(
    map(celsius_to_fahrenheit, celsius_values)
)

print(fahrenheit_again)

# Output:
# [32.0, 68.0, 86.0, 104.0]
```

The parameter name can be temp, T, or another valid name. What matters is that the function consistently uses its own parameter inside the function body.

6.5 map() vs Loop

Both can transform data. `map()` is useful when the operation is naturally 'apply this function to every item.' A normal loop is often clearer when the logic has several steps or side effects.

7. Lambda Functions

A **lambda** is a small anonymous function written as one expression. It is useful for short operations passed directly to functions such as `map()` and `filter()`.

```
lambda arguments: expression
```

7.1 Simple Example

```
add = lambda a, b: a + b

print(add(4, 3))

# Output:
# 7
```

7.2 Lambda with map()

```
numbers = [1, 2, 3, 4, 5, 6]

result = list(
    map(lambda x: x + 1, numbers)
)

print(result)

# Output:
# [2, 3, 4, 5, 6, 7]
```

For each value x , the lambda calculates $x + 1$. For complex or reusable logic, prefer a normal `def` function.

8. filter()

8.1 Meaning

`filter()` keeps only elements for which a function returns a truthy result.

```
def is_even(number):
    return number % 2 == 0

numbers = [1, 2, 3, 4, 5, 6, 7, 8]

result = list(
    filter(is_even, numbers)
)

print(result)

# Output:
# [2, 4, 6, 8]
```

8.2 filter() with Lambda

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]

even_numbers = list(
    filter(lambda x: x % 2 == 0, numbers)
)

print(even_numbers)

# Output:
# [2, 4, 6, 8]
```

8.3 Filtering Words by Length

```
words = ["Python", "AI", "Machine", "Data", "Science"]

long_words = list(
    filter(lambda word: len(word) >= 6, words)
)
```



```
print(long_words)

# Output:
# ['Python', 'Machine', 'Science']
```

8.4 map() vs filter()

map()	filter()
Transforms every input	Selects inputs using a condition
Returns each function result	Returns original items that pass
Example: square every number	Example: keep only even numbers

8.5 Boolean Results vs Filtered Values

```
numbers = [1, 2, 3, 4]

checks = list(
    map(lambda x: x % 2 == 0, numbers)
)

evens = list(
    filter(lambda x: x % 2 == 0, numbers)
)

print(checks)
print(evens)

# Output:
# [False, True, False, True]
# [2, 4]
```

map() returns the result of the function for every item. filter() uses the function result as a decision and keeps only the original items that pass.

9. reduce() — Completing the Comparison

The class discussion introduces reduce conceptually. reduce() repeatedly combines items until one accumulated result remains. In Python 3, import it from functools.

```
from functools import reduce

numbers = [1, 2, 3, 4]

total = reduce(
    lambda accumulator, value: accumulator + value,
    numbers
)

print(total)

# Output:
# 10
```

Conceptually: $1 + 2 = 3$, then $3 + 3 = 6$, then $6 + 4 = 10$.

Function	Purpose	Example
map()	Transform every item	$[1,2,3] \rightarrow [1,4,9]$

Function	Purpose	Example
filter()	Keep matching items	[1,2,3,4] → [2,4]
reduce()	Combine to one result	[1,2,3,4] → 10

10. Complete Combined Example

```

from functools import reduce

numbers = [1, 2, 3, 4, 5, 6]

# FILTER: keep even numbers
evens = list(
    filter(lambda x: x % 2 == 0, numbers)
)

# MAP: square each even number
squares = list(
    map(lambda x: x ** 2, evens)
)

# REDUCE: add all squared values
total = reduce(
    lambda a, b: a + b,
    squares
)

print(evens)
print(squares)
print(total)

# Output:
# [2, 4, 6]
# [4, 16, 36]
# 56

```

The pipeline is: **filter** → **transform** → **aggregate**.

11. Common Mistakes and Important Corrections

1. **print is not return**: print displays; return sends a reusable result.
2. **No explicit return means None**: Python functions return None by default.
3. **A generator is not memory-free**: it stores execution state, but usually not the entire output sequence.
4. **map() does not filter**: it applies a function and returns each function result.
5. **filter() returns original items that pass the condition**.
6. **map() and filter() are lazy in Python 3**: use list(...) when a list is required.
7. **Lambda should be short**: use def for complex or reusable logic.
8. **Do not force map/filter everywhere**: comprehensions or normal loops can be clearer.

12. Quick Interview Questions

What is a function?

A reusable block of code that performs a specific task and can accept inputs and return outputs.

Parameter vs argument?

A parameter appears in the function definition; an argument is the actual value passed during a call.

print vs return?

print displays a value; return sends a value back for storage or reuse.

What is an iterable?

An object whose elements can be iterated over.

What is an iterator?

An object that returns one item at a time and remembers its iteration state.

What is a generator?

A special iterator commonly created with yield that lazily produces values.

What does map() do?

Applies a function to each item.

What does filter() do?

Keeps items whose predicate is truthy.

What does reduce() do?

Combines a sequence into one accumulated result.

What is lambda?

A small anonymous function consisting of one expression.

13. Final Cheat Sheet

```
# FUNCTION
def add(a, b):
    return a + b

# ITERATOR
iterator = iter([10, 20, 30])
first = next(iterator)

# GENERATOR
def squares(n):
    for x in range(n):
        yield x ** 2

# MAP
mapped = list(
    map(lambda x: x * 2, [1, 2, 3])
)

# FILTER
filtered = list(
    filter(lambda x: x % 2 == 0, [1, 2, 3, 4])
)

# REDUCE
from functools import reduce

total = reduce(
    lambda a, b: a + b,
```

```
[1, 2, 3, 4]  
)
```

Recommended learning order: Functions → parameters and arguments → return → iterable → iterator → generator/yield → map → lambda → filter → reduce. This sequence makes the relationship between the topics easier to understand.